AGENT DESIGN SPEC: Phase Gate Validator

1. JOB TO BE DONE

One job, done well: The agent validates phase advancement requests in a video editing project workflow by intelligently assessing checklist completion, historical patterns, and contextual factors—then recommends approval, conditional approval, or human escalation.

Why this job: The "Manage Workflow/Phase Progress" process (shown in my Level 2 DFD) is currently a deterministic workflow: Receive Request → Validate Gate (Checklist Status) → Update Phase State. This works for standard cases but fails when:

•Checklist items are partially complete
•Exceptions need evaluation
•Historical patterns suggest hidden risks
•Edge cases require nuanced judgment

An agent can handle these ambiguous situations that a rules-based system cannot.

Scope boundary: The agent does NOT execute phase advancement. It assesses, recommends, and escalates. The final decision (and the actual state update) remains with a human approver. This keeps the agent low-risk while delivering maximum value.

---

2. THE USER & USAGE FREQUENCY

Element	Detail
User	Me (Shoaib Nagy), full-stack engineer building the platform
Usage frequency	Daily during development (5–10 phase advance requests per day during active development)
What success	The agent saves me 2–3 hours per week by automating checklist

Element	Detail
looks like	validation and surfacing issues early, allowing me to focus on building rather than gatekeeping

---

3. TOOLS AND DATA NEEDED

Data Sources (What the agent reads)

Data	Source	Access Plan
Phase progress data	PostgreSQL (Phase Progress table)	MCP tool: `query_phase_progress` (read-only SELECT)
Checklist items	PostgreSQL (Checklist Items table)	MCP tool: `query_checklist_status` (read-only SELECT)
Phase history	PostgreSQL (Phase History table)	MCP tool: `query_phase_history` (read-only SELECT)
Project context	Repository README + system design docs	MCP resource: `project_context.md` (static file)

Tools (What the agent does)

Tool	Type	Description
`query_phase_progress`	Data	Retrieve current phase state for a project
`query_checklist_status`	Data	Retrieve completion status of all checklist items for a phase
`query_phase_history`	Data	Retrieve historical phase advancement patterns
`check_phase_dependencies`	Data	Check if prerequisite phases are complete
`recommend_approval`	Action	Generate a structured recommendation (approve / conditional / escalate)
`escalate_to_human`	Action	Create an escalation request with context for human review

Access Plan

All database queries use **read-only connections** with a dedicated database user that has SELECT privileges only. The agent has no write access to the database. This is a non-negotiable guardrail—the agent recommends, it does not execute.

---

4. FIVE EVAL CASES

#	Scenario	Expected Outcome
1	All checklist items complete, all dependencies satisfied. Phase "Design Review" → "Implementation"	**APPROVE** — agent should recommend approval with brief reasoning
2	One non-critical checklist item incomplete. Phase "Implementation" → "Testing"	**CONDITIONAL** — agent should recommend conditional approval, specify the missing item and what to do about it
3	A mandatory checklist item incomplete. Phase "Requirements" → "Design"	**ESCALATE** — agent should escalate to human because mandatory items must be complete
4	Phase advance requested but a prerequisite phase is still in progress	**CONDITIONAL** or **ESCALATE** — agent should detect the dependency and recommend resolving it first
5	History shows the same phase was rolled back twice in the past month for similar projects	**ESCALATE** — agent should flag the historical pattern and recommend human review before approving

---

5. RISKS AND GUARDRAILS

What the Agent Must Confirm

Action	Confirmation Required
Before making any recommendation	All data queries returned successfully and the state is current
Before recommending APPROVE	All mandatory checklist items are confirmed complete, all dependencies are satisfied, and no red flags exist in phase history
Before ESCALATING	The escalation includes full context (checklist status, dependency status, history) so the human can make an informed decision

What the Agent Must NEVER Do

Prohibition	Rationale
Write to the database	The agent has read-only access only. State updates are performed by the human or the workflow system, not the agent.
Auto-approve without reviewing checklists	The agent must always query and review checklist status before making a recommendation. No shortcuts.
Escalate without evidence	Every escalation must include specific reasoning and data. No "I think this is bad" without justification.
Recommend approval for a phase with incomplete mandatory items	This is the single most critical guardrail. Mandatory items are non-negotiable.
Act without context	The agent must always have access to the current phase state, checklist status, and history before making a recommendation.

Human Intervention Triggers

Trigger	Action
Agent exceeds 5 sequential escalations in a row	Investigate whether the agent is too conservative or the process has systemic issues
Agent recommends APPROVAL for a phase that later fails QA	Human reviews the agent's reasoning to identify gaps in the evaluation logic
User requests manual override	Human can always override the agent's recommendation—the agent is advisory, not authoritative

---

6. BUILD PLATFORM CHOICE

Chosen Platform: Claude Project with Connectors and Skills

Element	Implementation
Model	Claude 3.5 Sonnet (available in my Claude Project)
Tools	MCP tools defined as skills (read-only database queries, recommendations)
Instructions	Project custom instructions (the draft instructions above)
Evaluation	Manual testing against the 5 eval cases before production use
Cost	Free tier (I already have a Claude Project from earlier assignments)

Justification

Alternative	Why I Chose This Instead
Custom GPT (OpenAI)	Requires ChatGPT Plus subscription ($20/mo) and has limited tool support for PostgreSQL. Also, the PDF guide emphasizes that agents need well-defined tools—OpenAI's custom GPT tool support is less flexible than MCP.
n8n Agent Workflow	n8n is powerful but requires setting up a full workflow engine. My use case is simple enough (read database → reason → recommend) that a Claude

Alternative	Why I Chose This Instead
	Project with MCP tools is more direct and maintainable. n8n would add unnecessary complexity.

Why Claude Project wins:

•I already have a Claude Project set up from previous assignments
•MCP connectors can access PostgreSQL via read-only queries
•The Project's custom instructions provide the structured guidance needed
•No additional cost or platform learning curve
•My FL-04 pipeline is already designed as a workflow—this agent is the natural next step